

METU Computer Engineering

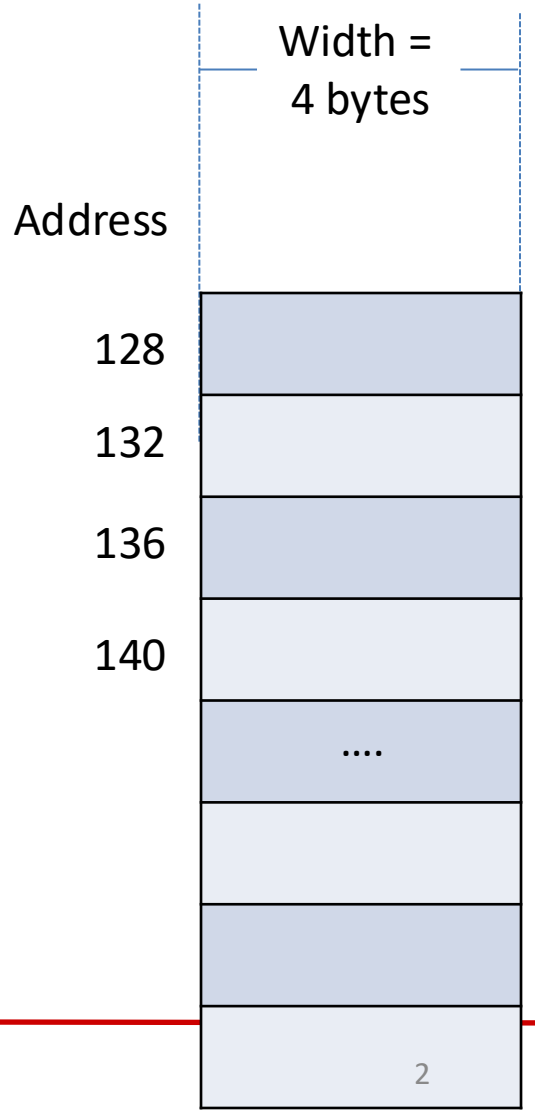
CEng 111 – Fall 2026
Week 7a

Credit: Some slides are from the “Invitation to Computer Science” book by G. M. Schneider, J. L. Gersting and some from the “Digital Design” book by M. M. Mano and M. D. Ciletti.

Previously on CENG111!

Structured Data

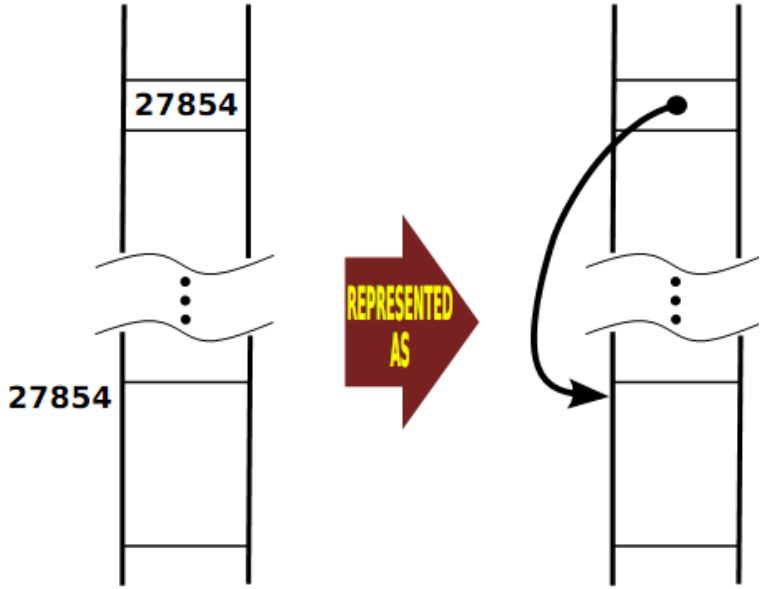
- If you have lots and lots of one type of data (for example, the ages of all the people in Turkey):
  - You can store them into memory consecutively (supported by most PLs)
    - This is called *arrays*.
  - Easy to access an element. Nth element:
    - <Starting-address>+ (N-1)*<Word Width>
    - Ex: 2nd element is at 128 + (2-1) * 4 = 132

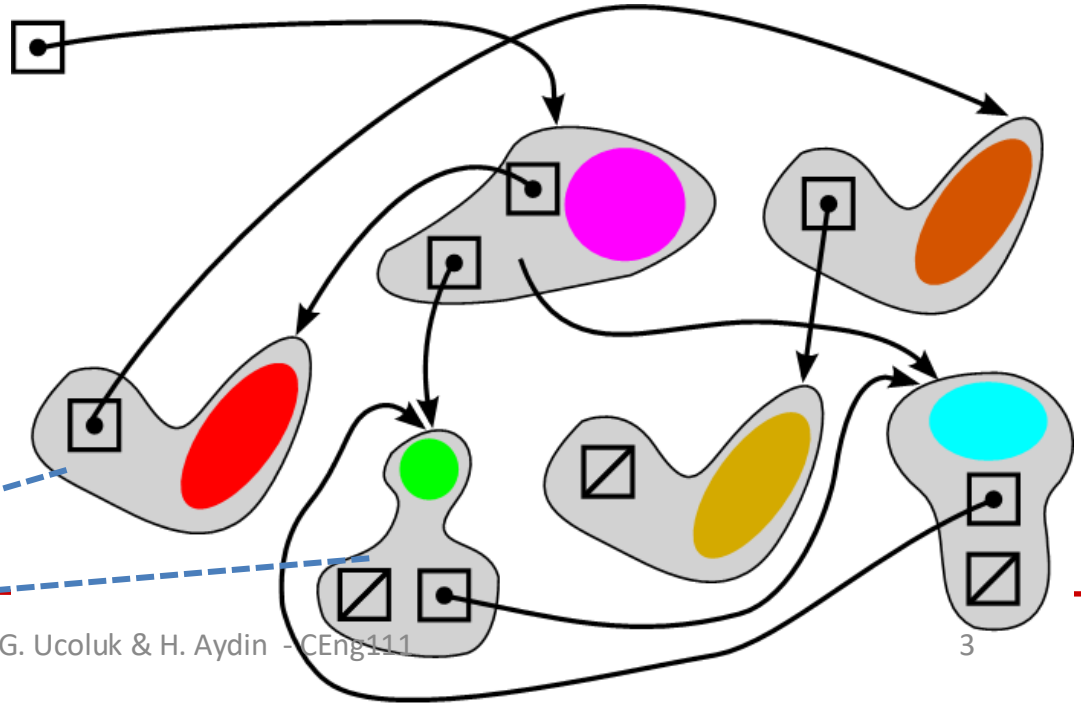


Structured Data

Previously on CENG111!

- What if you have to make a lot of deletions and insertions in the middle of an array?
- Then, you have to store your data in blocks/units such that each unit has the starting address of the next unit/block.





Blocks of Data



Previously on CENG111!

Strings in Python

Python provides the **str** data type for strings:

```
>>> "Hello?"
'Hello?'
>>> type("Hello?")
<type 'str'>
```

- Simplest operation with a string:

```
>>> len("Hello?")
6
```

Previously on CENG111!

Strings in Python

- Accessing elements of a string
  - "Hello?"[0] → 1st character (i.e., "H")
  - "Hello?"[4] → 5th character
- Indexing starts at 0!!!
- What is the last element then?
  - "Hello?"[len("Hello?") - 1]
- Negative indexing possible:
  - Last element: "Hello?"[-1] → "?"
- In general:
  - String[start:end:step]
  - Ex: "Hello?"[0:4:2] → "Hl"
  - Ex: "Hello?"[2:4] → "ll"



Creating Strings in Python

Previously on CENG111!

1. Enclosing a set of characters between quotes:
   - “ali”, “veli”, “deli”, …
2. Using the str() function:
   - str(4.5) → “4.5”
3. Using the raw_input() function:

```
>>> a = raw_input("--> ")
--> Do as I say
>>> a
'Do as I say'
>>> type(a)
<type 'str'>
```

Python v2

```
>>> a = input("--> ")
--> Do as I say
>>> a
'Do as I say'
>>> type(a)
<class 'str'>
>>>
```

Python v3



Previously on CENG111!

Tuples in Python

```
>>> (1, 2, 3, 4, "a")
(1, 2, 3, 4, 'a')
>>> type((1, 2, 3, 4, "a"))
<type 'tuple'>
```

- Tuples in Python: collection of data enclosed in parentheses, separated by comma.
- Accessing elements of a tuple (like strings):
  - Positive Indexing: `(1, 2, 3, 4, "a")[2]` returns 3.
  - Negative Indexing: `(1, 2, 3, 4, "a")[-1]` returns 'a'.
  - Ranged Indexing, *i.e.*, `[start:end:step]`: `(1, 2, 3, 4, "a")[0:4:2]` leads to `(1, 3)`.



Previously on CENG111!

Creating Tuples in Python

1. Enclosing data within parentheses:
   - Ex: (1, “a”, “cde”, 23)
2. Using the tuple() function:
   - Ex: tuple(“ABC”) → (‘A’, ‘B’, ‘C’)
3. Using the input() function:

```
>>> a = input("Give me a tuple:")
Give me a tuple:(1, 2, 3)
>>> a
(1, 2, 3)
>>> type(a)
<type 'tuple'>
```

Python v2

```
>>> a = eval(input("Give me a tuple:"))
Give me a tuple:(1,2,3)
>>> a
(1, 2, 3)
>>> type(a)
<class 'tuple'>
```

Python v3



Today

- List data type
- Naming data: Variables
- Aliasing

METU Computer Engineering

Administrative Notes

- Midterm Exam
  - Date: 20 November, 18:00
- Lab Exams
  - Lab Exam 1: Held on 8 November, Sunday, 13:00
  - Lab Exam 2: 6 December, Sunday, 13:00
  - Lab Exam 3: 20 December, Sunday, 13:00
  - Lab Exam 4: 3 January, Sunday, 13:00



Lists

- Similar to tuples.
- Difference:
  - Tuples are immutable (i.e., not changeable) whereas lists are mutable.

Lists in Python

```
>>> [1, 2, 3, 4, "a"]
[1, 2, 3, 4, 'a']
>>> type([1, 2, 3, 4, "a"])
<type 'list'>
```

- Lists in Python: collection of data enclosed in brackets, separated by comma.
- Accessing elements of a list (like strings & tuples):

- Positive Indexing: `[1, 2, 3, 4, "a"][2]` returns 3.
- Negative Indexing: `[1, 2, 3, 4, "a"][-1]` returns `'a'`.
- Ranged Indexing, *i.e.*, `[start:end:step]`: `[1, 2, 3, 4, "a"][0:4:2]` leads to `[1, 3]`.

Creating Lists in Python

METU Computer Engineering

1. Enclosing data within brackets:
   - Ex: [1, “a”, “cde”, 23]
2. Using the list() function:
   - Ex: list(“ABC”) → [‘A’, ‘B’, ‘C’]
3. Using the range() function: `range( [start,] stop[, step])`
   - Ex: range(1, 10, 2) → [1, 3, 5, 7, 9]
4. Using the input() function:

```
>>> a = input("Give me a list:")
Give me a list:[1, 2, "a"]
>>> a
[1, 2, 'a']
>>> type(a)
<type 'list'>
```

Python v2

```
>>> a = eval(input("Give me a list:"))
Give me a list:[1,2,"a"]
>>> a
[1, 2, 'a']
>>> type(a)
<class 'list'>
```

Python v3

Modifying a List in Python

- List[range] = Data
  - Ex:

```
>>> L = [3, 4, 5, 6, 7, '8', 9, '10']
>>> L[::2]
[3, 5, 7, 9]
>>> L[::2] = [4, 6, 8, 10]
>>> L[::2]
[4, 6, 8, 10]
>>> L[]
[4, 4, 6, 6, 8, '8', 10, '10']
```

- Using the append() function:
  - List.append(item)
  - Ex: [1, 2, 3].append(5) → [1, 2, 3, 5]

Modifying a List in Python

- Using the extend() function:
  - List.extend(Another_list)
  - Ex: [1, 2, 3].extend([“a”, “b”]) → [1, 2, 3, “a”, “b”]

```
>>> L.extend(["a", "b"])
>>> L
[4, 4, 6, 6, 8, '8', 10, '10', 'a', 'a', 'b']
```

- Using the insert() function:
  - List.insert(index, item)

```
>>> L=[1, 2, 3]
>>> L
[1, 2, 3]
>>> L.insert(1, 0)
>>> L
[1, 0, 2, 3]
```

Removing Elements from a List in Python

- **del statement:** `del L[start:end]`

```
>>> L
[1, 0, 2, 3]
>>> del L[1]
>>> L
[1, 2, 3]
```

- **L.pop() function:** `L.pop([index])`

```
>>> L=[1,2,3]
>>> L.pop()
3
>>> L
[1, 2]
>>> L.pop(0)
1
>>> L
[2]
```

- **L.remove() function:** `L.remove(value)`

```
>>> L
[2, 1, 3]
>>> L.remove(1)
>>> L
[2, 3]
```



A frequent operation with containers

METU Computer Engineering

- Membership
  - in
  - not in
- “en” in “deneme”
  - True
- “an” in “deneme”
  - False
- “dem” in “deneme”
  - False



Accessing Data/Containers by Names: Variables

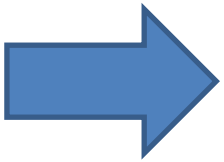
METU Computer Engineering

- Naming:
  - Usually: A combination of letters and numbers
  - Ex: a123, 123a, …
- Scope & Extent:
  - Scope: Where a variable can be accessed.
  - Extent: The lifetime of a variable.
- Typing:
  - Statically typed: The type of a variable is fixed.
  - Dynamically typed: The type of a variable is variable ☺

Variables in Python

METU Computer Engineering

```
>>> a = 4
>>> b = 3
>>> c = a + b
>>> a
4
>>> b
3
>>> c
7
```



- We don’t need to declare a variable before using it.
- We don’t need to specify the type of a variable.

- ‘=‘ means “Change the content of the variable with the value at the right-hand side”.
  - Assignment!
- The left-side of the assignment should be a valid variable name:
  - Ex: a+2 = 5 → NOT VALID!



Variable Naming in Python

v2:

- Variable names are case sensitive. So, the names `a` and `A` are two different variables.
- Variable names can contain letters from the English alphabet, numbers and an underscore `_`.
- Variable names can only start with a letter or an underscore. So, `10a`, `$a`, and `var$` are all invalid whereas `_a` and `a_20`, for example, are valid names in Python.



Variable Naming in Python

v3:

- Variable names can contain Unicode characters.
- A name can start with a valid an **ID_Start** character and continue with an **ID_Continue** character:

From PEP3131 (PEP: Python Enhancement Proposal):

- **ID_Start** is defined as all characters having one of the general categories uppercase letters (Lu), lowercase letters (Ll), titlecase letters (Lt), modifier letters (Lm), other letters (Lo), letter numbers (Nl), the underscore, and characters carrying the Other_ID_Start property. XID_Start then closes this set under normalization, by removing all characters whose NFKC normalization is not of the form ID_Start ID_Continue* anymore.
- **ID_Continue** is defined as all characters in ID_Start, plus nonspacing marks (Mn), spacing combining marks (Mc), decimal number (Nd), connector punctuations (Pc), and characters carrying the Other_ID_Continue property.

For a full list of all characters for these categories, see:
https://web.archive.org/web/20081016132748/http://www.dcl.hpi.uni-potsdam.de/home/loewis/table-3131.html

ID_Start characters in Python v3

Category / Rule	Description	Example Character	Unicode Codepoint & Name
Uppercase Letters (Lu)	Standard uppercase alphabetic letters	A, Z, Ω	U+0041 (LATIN CAPITAL LETTER A)
Lowercase Letters (Ll)	Standard lowercase alphabetic letters	a, z, σ	U+0061 (LATIN SMALL LETTER A)
Titlecase Letters (Lt)	Digraph characters capitalized at the start	Dž	U+01C5 (LATIN CAPITAL LETTER D WITH SMALL LETTER Z)
Modifier Letters (Lm)	Letter-like symbols modifying adjacent sounds	ʰ, ˌ	U+02B0 (MODIFIER LETTER SMALL H)
Other Letters (Lo)	Syllabic scripts or logographic characters	中, あ, א	U+4E2D (CJK UNIFIED IDEOGRAPH-4E2D)
Letter Numbers (Nl)	Numeric values written using letterforms	Ⅴ (Roman 5), Ⅻ	U+2164 (ROMAN NUMERAL FIVE)
Underscore	The literal ASCII low line character	_	U+005F (LOW LINE)
Other_ID_Start	Non-letter characters explicitly included for backward compatibility	℘ (Script P), ℮	U+2118 (SCRIPT CAPITAL P)



Variable Naming in Python

- Variable names cannot be one of the keywords in Python:

and	del	from	not	while
as	elif	global	or	with
assert	else	if	pass	yield
break	except	import	print *	
class	exec	in	raise	
continue	finally	is	return	
def	for	lambda	try	



More on Variables in Python

- Typing of variables:
  - Python is dynamically typed:

```
>>> a = 3
>>> type(a)
<type 'int'>
>>> a = 3.4
>>> type(a)
<type 'float'>
```

- Using variables:

```
>>> a = (1, 2, 3, 'a')
>>> type(a)
<type 'tuple'>
>>> a[1]
2
>>> a[-1]
'a'
```